Kernel: Python 3 (CoCalc)

```
import numpy as np
import pandas as pd
```

```
human_interaction=pd.read_csv('personality_datasert.csv')
human_interaction
```

Out[4]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Dra
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	No
2896	3.0	No	8.0	3.0	No
2897	4.0	Yes	1.0	1.0	Yes
2898	11.0	Yes	1.0	3.0	Yes
2899	3.0	No	6.0	6.0	No

2900 rows × 8 columns

Dataset Information

The title of this dataset is Human Interaction, because we're exploring, different factors, such as, alone time, stage fear, how frequent they attend social events, how frequent they go outside, if they feel drained after socializing, the amount of friends in their circle, and how frequent they post on social media, that contribute to people's personality (extrovert vs introvert).

Human Interation

The primary purpose of this data is to show how a person's personality influnces their daily habits. By looking at these patterns, we can understand why some people need more social time while others need more time alone to recharge their energy.

Basic data exploration

```python
print(f'The first 5 rows of the dataset:')
human_interaction.head(5)
```

Out[5]: The first 5 rows of the dataset:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Drained
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No

```python
print(f'The last 5 rows of the dataset:')
human_interaction.tail(5)
```

Out[6]: The last 5 rows of the dataset:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Dra
2895	3.0	No	7.0	6.0	No
2896	3.0	No	8.0	3.0	No
2897	4.0	Yes	1.0	1.0	Yes
2898	11.0	Yes	1.0	3.0	Yes
2899	3.0	No	6.0	6.0	No

```python
print(f'There are {human_interaction.shape} rows , columns in the
dataset')
print(f'The names of the columns are; {human_interaction.columns}')
```

Out[7]:
```
There are (2900, 8) rows , columns in the dataset
The names of the columns are; Index(['Time_spent_Alone', 'Stage_fear',
'Social_event_attendance',
       'Going_outside', 'Drained_after_socializing',
'Friends_circle_size',
       'Post_frequency', 'Personality'],
      dtype='object')
```

Data Types and Structure

```python
human_interaction.info()
```

Out[8]:
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2900 entries, 0 to 2899
Data columns (total 8 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   Time_spent_Alone           2900 non-null   float64
 1   Stage_fear                 2900 non-null   object 
 2   Social_event_attendance    2900 non-null   float64
 3   Going_outside              2900 non-null   float64
 4   Drained_after_socializing  2900 non-null   object 
 5   Friends_circle_size        2900 non-null   float64
 6   Post_frequency             2900 non-null   float64
 7   Personality                2900 non-null   object 
dtypes: float64(5), object(3)
memory usage: 181.4+ KB
```

```
human_interaction.select_dtypes(include=['object'])
```

Out[9]:

	Stage_fear	Drained_after_socializing	Personality
0	No	No	Extrovert
1	Yes	Yes	Introvert
2	Yes	Yes	Introvert
3	No	No	Extrovert
4	No	No	Extrovert
...	...	...	...
2895	No	No	Extrovert
2896	No	No	Extrovert
2897	Yes	Yes	Introvert
2898	Yes	Yes	Introvert
2899	No	No	Extrovert

2900 rows × 3 columns

```
human_interaction.select_dtypes(include=['number'])
```

Out[10]:

	Time_spent_Alone	Social_event_attendance	Going_outside	Friends_circle_
0	4.0	4.0	6.0	13.0
1	9.0	0.0	0.0	0.0
2	9.0	1.0	2.0	5.0
3	0.0	6.0	7.0	14.0
4	3.0	9.0	4.0	8.0
...	...	...	...	...
2895	3.0	7.0	6.0	6.0
2896	3.0	8.0	3.0	14.0
2897	4.0	1.0	1.0	4.0
2898	11.0	1.0	3.0	2.0
2899	3.0	6.0	6.0	6.0

2900 rows × 5 columns

The Stage_fear and Drained_after_socializing columns should be converted from categorical to numerical in order to check if there is a correlation analysis between all the data that explains the person personality type

Descriptive Summary

```
hi_summary=human_interaction.describe()
print(hi_summary)
hi_mean=human_interaction['Time_spent_Alone'].mean()
print(hi_mean)
hi_median=human_interaction['Time_spent_Alone'].median()
print(hi_median)
hi_range=human_interaction['Time_spent_Alone'].max()+human_interaction[
'Time_spent_Alone'].min()
print(hi_range)
hi_std=human_interaction['Time_spent_Alone'].std()
print(hi_std)
```

Out[11]:
```
       Time_spent_Alone  Social_event_attendance  Going_outside  \
count       2900.000000              2900.000000    2900.000000
mean           4.505816                 3.963354       3.000000
std            3.441180                 2.872608       2.221597
min            0.000000                 0.000000       0.000000
25%            2.000000                 2.000000       1.000000
50%            4.000000                 3.963354       3.000000
75%            7.000000                 6.000000       5.000000
max           11.000000                10.000000       7.000000

       Friends_circle_size  Post_frequency
count          2900.000000     2900.000000
mean              6.268863        3.564727
std               4.232340        2.893587
min               0.000000        0.000000
25%               3.000000        1.000000
50%               5.000000        3.000000
75%              10.000000        6.000000
max              15.000000       10.000000
4.50581600281988
4.0
11.0
3.4411804188954207
```

The mean size of a friend circle is approximately 6.27, but the standard deviation of 4.23 tells us there is a high degree of variability in the data. While the "typical" person has about 6 friends, some individuals report having 0 friends (minimum), while others have up to 15 friends (maximum). This wide range suggests that the dataset captures diverse social behaviors, likely categorized by the personality types (Introverts vs. Extroverts). For most columns like Social_event_attendance and Going_outside the mean and median are very close. This shows that the data for these behaviors is relatively symmetric and not heavily skewed by extreme outliers. There are no negatie valus or missing values in the numerical summaries.

Missing or Duplicate Data

```
missing_value=human_interaction.isnull().sum()
print(f'There are {missing_value} duplicated values in each column in human interaction')
```

Out[12]:
```
There are Time_spent_Alone             0
Stage_fear                  0
Social_event_attendance     0
Going_outside               0
Drained_after_socializing   0
Friends_circle_size         0
Post_frequency              0
Personality                 0
dtype: int64 duplicated values in each column in human interaction
```

```
duplicate=human_interaction.duplicated().sum()
print(f'There are {duplicate} duplicated rows in human interaction')
```

Out[13]:
```
There are 402 duplicated rows in human interaction
```

Data Wrangling

Adding a new column based on another

```
hi_new=human_interaction.copy()
hi_new
```

Out[14]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Dra
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	No
2896	3.0	No	8.0	3.0	No
2897	4.0	Yes	1.0	1.0	Yes
2898	11.0	Yes	1.0	3.0	Yes
2899	3.0	No	6.0	6.0	No

2900 rows × 8 columns

```
def drained(Time_spent_Alone):
    if pd.isnull(Time_spent_Alone):
        return 'N/A'
    elif Time_spent_Alone < 5:
        return 1
    else:
        return 2

def fright(Friends_circle_size):
    if pd.isnull(Friends_circle_size):
        return 'N/A'
    elif Friends_circle_size <= 5:
        return 2
    else:
        return 1
```

```
hi_new['Drained_after_events']=hi_new['Time_spent_Alone'].apply(drained
)
hi_new['Stage_fright']=hi_new['Friends_circle_size'].apply(fright)
hi_new
```

Out[16]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Dra
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	No
2896	3.0	No	8.0	3.0	No
2897	4.0	Yes	1.0	1.0	Yes
2898	11.0	Yes	1.0	3.0	Yes
2899	3.0	No	6.0	6.0	No

2900 rows × 10 columns

```
hi_new.head(11)
```

Out[17]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Drain
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No
5	1.0	No	7.0	5.0	No
6	4.0	No	9.0	3.0	No
7	2.0	No	8.0	4.0	No
8	10.0	Yes	1.0	3.0	Yes
9	0.0	No	8.0	6.0	No
10	3.0	No	9.0	6.0	No

The new column Drained after events is based on the person's rating of how much time they spend alone. If they spend less than 5 hours alone, they're considered extroverts who rarely feel drained after socializing. So those who are considered extroverts are given the number 1, in case a correlation table is needed later on. And if they have 5 or less friends they are consdered introverts, so they are given the number 2.

Filtering the Data

```
hi_filtered1=hi_new.copy()
hi_filtered1
```

Out[18]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Dra
0	4.0	No	4.0	6.0	No
1	9.0	Yes	0.0	0.0	Yes
2	9.0	Yes	1.0	2.0	Yes
3	0.0	No	6.0	7.0	No
4	3.0	No	9.0	4.0	No
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	No
2896	3.0	No	8.0	3.0	No
2897	4.0	Yes	1.0	1.0	Yes
2898	11.0	Yes	1.0	3.0	Yes
2899	3.0	No	6.0	6.0	No

2900 rows × 10 columns

```
hi_filtered1.drop(columns=['Drained_after_socializing'], inplace=True)
print('\n\n Data after dropping Drained after socializing column:\n\n')
hi_filtered1
```

Out[19]:

```
 Data after dropping Drained after socializing column:
```

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Fri
0	4.0	No	4.0	6.0	13.
1	9.0	Yes	0.0	0.0	0.0
2	9.0	Yes	1.0	2.0	5.0
3	0.0	No	6.0	7.0	14.
4	3.0	No	9.0	4.0	8.0
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	6.0
2896	3.0	No	8.0	3.0	14.
2897	4.0	Yes	1.0	1.0	4.0
2898	11.0	Yes	1.0	3.0	2.0
2899	3.0	No	6.0	6.0	6.0

2900 rows × 9 columns

```
hi_filtered=hi_filtered1.copy()
hi_filtered
```

Out[20]:

	Time_spent_Alone	Stage_fear	Social_event_attendance	Going_outside	Fri
0	4.0	No	4.0	6.0	13.
1	9.0	Yes	0.0	0.0	0.0
2	9.0	Yes	1.0	2.0	5.0
3	0.0	No	6.0	7.0	14.
4	3.0	No	9.0	4.0	8.0
...	...	...	...	...	...
2895	3.0	No	7.0	6.0	6.0
2896	3.0	No	8.0	3.0	14.
2897	4.0	Yes	1.0	1.0	4.0
2898	11.0	Yes	1.0	3.0	2.0
2899	3.0	No	6.0	6.0	6.0

2900 rows × 9 columns

```
hi_filtered.drop(columns=['Stage_fear'], inplace=True)
print('\n\n Data after dropping Stage fear column:\n\n')
hi_filtered
```

Out[21]:

```
 Data after dropping Stage fear column:
```

	Time_spent_Alone	Social_event_attendance	Going_outside	Friends_circle_
0	4.0	4.0	6.0	13.0
1	9.0	0.0	0.0	0.0
2	9.0	1.0	2.0	5.0
3	0.0	6.0	7.0	14.0
4	3.0	9.0	4.0	8.0
...	...	...	...	...
2895	3.0	7.0	6.0	6.0
2896	3.0	8.0	3.0	14.0
2897	4.0	1.0	1.0	4.0
2898	11.0	1.0	3.0	2.0
2899	3.0	6.0	6.0	6.0

2900 rows × 8 columns

```
hi_filtered.to_csv('Personality_Dataset_filtered.csv', index=False)
```

I filtered the data to make the stage fear column and drained after events column numerical and dropped the categorical columns.

Unique Values and Categories

```
unique_values=hi_new['Stage_fear'].unique()
print(f'Are there any unique values in the Stage fear column?
{unique_values}')
```

Out[23]:
```
Are there any unique values in the Stage fear column? ['No' 'Yes']
```

```
unique_values1=hi_new['Drained_after_socializing'].unique()
print(f'Are there any unique values in the Drained after socializing
column? {unique_values1}')
```

Out[24]:
```
Are there any unique values in the Drained after socializing column?
['No' 'Yes']
```

```
unique_values2=hi_new['Personality'].unique()
print(f'Are there any unique values in the Personalty column?
{unique_values2}')
```

Out[25]:
```
Are there any unique values in the Personalty column? ['Extrovert'
'Introvert']
```

Personality column is the best variable for grouping to see the big-picture differences, while the Drained_after_socializing and Stage_fear columns are better for correlation analysis to see how specific anxieties or energy levels impact a person's lifestyle

Gouping and Aggregation

```
personality_by_stage_flight=hi_filtered.groupby('Personality').agg({
    'Time_spent_Alone': 'mean',
    'Friends_circle_size': 'mean',
    'Social_event_attendance': 'mean'
})
personality_by_stage_flight
```

Out[26]:

	Time_spent_Alone	Friends_circle_size	Social_event_attendance
Personality			
Extrovert	2.122869	9.095744	5.977850
Introvert	7.027444	3.277465	1.831621

For this data, I learned that there is a distinction in lifestyle hbit between the two personality types. Intoverts spend signiicantly more time alone an have similar friend circles, while Extroverts attend nearly three times as many social events and maintain larger social networks on average.

Summary and Conclusion

One interesting aspect of this dataset is the almost textook contrast between how introverts and extroverts manage their social energy and time. A clear pattern appeared during the initial exploration: introverts report spending over three times as much time alone as extroverts, while extroverts maintain significantly larger friend circles and attend more social events. However, a few questions arise regarding the data's distribution, particularly, the Friends_circle_size column shows some people with zero friends maintain a high Post_frequency on social media, which suggests that digial social engagement doesn't always mirror physical social circles. Additionally, the lack of 'neutral' or 'Ambivert' categories rises the question of whether the data was intentionally simplfied for classification puroses or if individuals were forced to choose between those two options.